

Relatório de Auditoria de Segurança

App Newflexo

Data:	01/09/2026
Escopo:	Aplicação Next.js (App Router), server actions e rotas /api, camada Prisma/Postgres, frontend React.
Stack:	Next.js 16 · React 19 · Prisma 7 + adapter-pg (Neon Postgres) · bcryptjs · Zod · TailwindCSS.

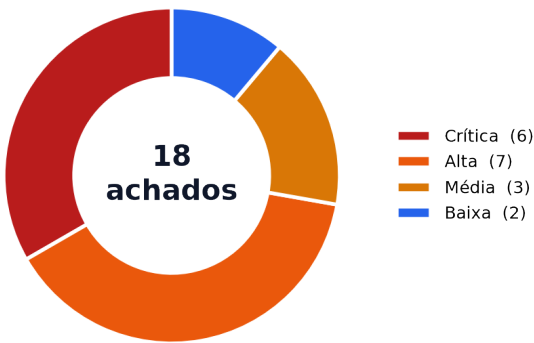
Nota metodológica. As cinco categorias solicitadas foram mapeadas para esta stack assim: (1) *Isolamento de inquilino* → o projeto não usa RLS; o isolamento é um filtro manual por *vendedorId* resolvido em *getRequesterVendedorId* — avaliou-se onde ele falta ou é contornável. (2) *Permissão no navegador* → cruzamento dos gates de UI (*isAdmin*) com a checagem equivalente nas server actions/rotas. (3) *IDOR* → varredura de todos os handlers que operam por id. (4) *Chaves expostas* → código, .env, scripts e histórico do git. (5) *XSS* → sinks *dangerouslySetInnerHTML/innerHTML* e renderização de conteúdo. Foram reportados apenas achados verificados no código real, com arquivo e linha.

1. Resumo executivo

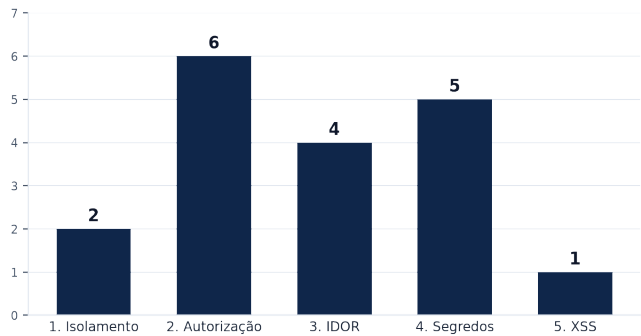
A auditoria identificou **18 achados**. O ponto central é arquitetural: **a aplicação não mantém sessão no servidor** — a identidade e o papel do usuário vêm do cliente (localStorage e um *requesterId* passado às server actions). Como consequência, os controles de isolamento por vendedor e as checagens de posse, embora existam no código, são **opcionais e falsificáveis**. Somam-se a isso operações administrativas sem verificação de papel no servidor e segredos reais (banco e chave de IA) versionados no git.



Distribuição por severidade



Achados por categoria



2. Pontos fortes e pontos fracos

2.1. Pontos fortes (o que está protegido)

- **SQL parametrizado em todo o código.** Todo o SQL bruto usa tagged templates do Prisma (`prisma.$queryRaw`...``) e `Prisma.sql/Prisma.join`. Valores entram como parâmetros (`$1`, `$2...`), não por concatenação. Não foi encontrada injeção de SQL mesmo com uso intenso de raw SQL (ex.: `clientes.ts`, `pedidos.ts`, `dashboard.ts`).
- **Senhas com hash bcrypt e nunca trafegadas.** Login usa `bcrypt.compare` (`app/api/auth/login/route.ts:25`) e o cadastro usa `bcrypt.hash`. As leituras que vão ao cliente excluem o campo 'senha' via `select` (`users.ts:68-73`, `verifySession` `users.ts:151-155`).
- **Escapamento automático do React nas telas.** A saída da IA e os dados do banco são renderizados como texto JSX (React escapa por padrão). Só há dois usos de `dangerouslySetInnerHTML` no projeto.
- **Uso de `dangerouslySetInnerHTML` controlado pelo desenvolvedor.** Em `components/ui/chart.tsx:83` o HTML injetado é CSS gerado a partir da config de cores do próprio código (não entra input do usuário) — uso correto do componente de gráfico.
- **Checação de posse já existe (parcial) em pedidos e orçamentos.** `getPedidoById/getOrcamentoById` e as escritas comparam `vendedorId` com a permissão do requisitante. A lógica está correta; o problema é ela ser opcional/bypassável, não inexistente.

2.2. Pontos fracos (riscos centrais)

- **Sem sessão no servidor.** A confiança recai sobre dados do cliente (`localStorage/requesterId`). Raiz dos demais riscos.
- **Autorização só no frontend.** Operações de admin e escrita não revalidam papel no servidor.
- **Isolamento contornável.** O filtro por vendedor é opcional; some quando o parâmetro não é enviado.
- **Segredos no repositório.** Credenciais do banco e chave de IA versionadas e no histórico do git.

3. Achados detalhados por categoria

1. Banco sem tranca (isolamento)

Sev.	Arquivo : linha	Descrição
Crítica	lib/actions/pedidos.ts 47, 61-63	[TEN-01] Filtro de isolamento por vendedor é opcional e controlado pelo cliente. O isolamento é um filtro manual por vendedorId derivado de getRequesterVendedorId(requesterId). Mas requesterId é opcional ('if (requesterId)'/'if (params.requesterId)') e vem do cliente. Omitindo requesterId, perm=null e nenhum filtro de vendedor é aplicado — retorna dados de TODOS os vendedores. Mesmo enviado, é o id que o cliente afirma ter (localStorage), falsificável. Mesmo padrão em getOrcamentos (orcamentos.ts:34-39), getOportunidadesData (oportunidades.ts:16-21), getComissoes (comissoes.ts:11-16) e getDashboardMetrics (dashboard.ts:16-21). <i>Impacto:</i> Vendedor (ou anônimo) enxerga pedidos, orçamentos, comissões e faturamento de toda a empresa.
Alta	lib/actions/clientes.ts 115-149	[TEN-02] Ficha do cliente expõe histórico de todos os vendedores. getClientById não recebe requesterId e retorna, além do cadastro, todos os orçamentos (121-126) e pedidos (127-132) do cliente, sem filtrar por vendedor. <i>Impacto:</i> Vazamento cruzado de negociações/valores entre vendedores concorrentes.

2. Permissão no navegador

Sev.	Arquivo : linha	Descrição
Crítica	lib/actions/users.ts 147-176 / lib/auth-context.tsx:44-135	[AUTH-01] Sessão inteiramente no cliente — servidor confia no userId enviado. Não há sessão no servidor (sem cookie assinado, sem token, sem middleware). A sessão vive em localStorage ('flexo_session') e a autenticação se resume a verifySession(userId) recebendo um userId escolhido pelo cliente. Qualquer pessoa pode gravar {userId: <id de um admin>} no localStorage, ou chamar as server actions diretamente, e ser tratada como aquele usuário. <i>Impacto:</i> Bypass total de autenticação e de papéis. É a causa-raiz que torna exploráveis os achados de isolamento (cat.1), autorização (cat.2) e IDOR (cat.3).
Crítica	lib/actions/users.ts 93, 127, 140	[AUTHZ-01] Gestão de usuários sem verificação de papel no servidor. saveUser (93), toggleUserActive (127) e updateUserPassword (140) não checam se quem chama é admin. A tela app/usuarios/page.tsx:80 esconde a UI com 'if (isAdmin)', mas a server action é um endpoint POST invocável diretamente. Um vendedor (ou anônimo) pode criar um usuário com role:'admin' ou redefinir a senha de qualquer conta. <i>Impacto:</i> Escalada de privilégio para admin e tomada de conta (account takeover) de qualquer usuário.
Alta	lib/actions/vendedores.ts 84, 110	[AUTHZ-02] Gestão de vendedores sem verificação de papel. saveVendedor (84) e toggleVendedorActive (110) não validam admin. UI protegida só em app/vendedores/page.tsx:64. <i>Impacto:</i> Criação/edição/inativação de vendedores por não-admin; adulteração de comissões (%).
Alta	lib/actions/config.ts 78, 117	[AUTHZ-03] Configuração da empresa e do módulo IA sem verificação de papel. updateEmpresa (78) e updateAIConfig (117) sem checagem de admin. UI protegida só em app/configuracoes/page.tsx:139. <i>Impacto:</i> Alteração de dados fiscais/cadastrais da empresa e da chave/limite de IA por não-admin.
Alta	lib/actions/creditos.ts 6, 49	[AUTHZ-04] Movimentação de crédito financeiro sem autorização. addMovimentacaoCredito (6) credita/debita saldo em R\$ e em etiquetas de qualquer cliente sem verificar quem chama nem vínculo. getMovimentacoesByCliente (49) idem para leitura. <i>Impacto:</i> Fraude financeira: crédito arbitrário para clientes; leitura do histórico financeiro alheio.

Sev.	Arquivo : linha	Descrição
Média	app/api/formas-pagamento/[id]/route.ts 4 (PUT), 31 (DELETE)	[AUTHZ-05] Rota REST de formas de pagamento sem autenticação. PUT e DELETE alteram/excluem formas de pagamento por id sem qualquer auth. UI protegida só em app/formas-pagamento/page.tsx:149. <i>Impacto:</i> Adulteração de condições de pagamento (parcelas) e exclusão de registros por qualquer um.

3. IDOR

Sev.	Arquivo : linha	Descrição
Crítica	lib/actions/clientes.ts 115, 170	[IDOR-01] Leitura e escrita de cliente por id sem verificação de posse. getClienteById(id) (115) devolve o cadastro completo, saldos de crédito e itens exclusivos de qualquer id. saveCliente(data) (170) atualiza qualquer cliente por id sem checagem. <i>Impacto:</i> Enumeração de toda a base de clientes e alteração de qualquer cadastro.
Alta	lib/actions/pedidos.ts 213, 246, 287, 346	[IDOR-02] Pedido por id: verificação de posse é bypassável. getPedidoById (213), updatePedidoStatus (246), cancelarPedido (287) e savePedido (346) só checam posse dentro de 'if (requesterId)'. Omitindo requesterId, a checagem inteira é pulada e qualquer pedido pode ser lido, alterado ou cancelado. <i>Impacto:</i> Acesso e manipulação de pedidos de outros vendedores (status, cancelamento, edição).
Alta	lib/actions/orcamentos.ts 181, 215, 270, 287	[IDOR-03] Orçamento por id: verificação de posse é bypassável. getOrcamentoById (181), updateOrcamentoStatus (215), deleteOrcamento (270) e saveOrcamento (287) seguem o mesmo padrão opt-in de requesterId — some quando o parâmetro não é enviado. <i>Impacto:</i> Leitura, alteração de status e exclusão lógica de orçamentos alheios.
Média	lib/actions/creditos.ts 49 / app/api/formas-pagamento/[id]/route.ts:31	[IDOR-04] Outros acessos por id sem posse (crédito e forma de pagamento). getMovimentacoesByCliente(clientId) (49) e o DELETE de formas de pagamento operam por id sem qualquer verificação de contexto do chamador. <i>Impacto:</i> Leitura de extrato financeiro por cliente e remoção de registros por id arbitrário.

4. Chaves expostas

Sev.	Arquivo : linha	Descrição
Crítica	.env 1	[SEC-01] Credenciais do banco (Neon Postgres) versionadas no git. DB_URL_OFFICIAL contém usuário e senha reais (npg...). O arquivo .env está rastreado no git (git ls-files confirma) e presente no histórico (commit 3c300a2, então como DATABASE_URL). O .gitignore ignora '.env*.local', mas não '.env'. <i>Impacto:</i> Quem tiver acesso ao repositório obtém acesso direto de leitura/escrita ao banco de produção.
Crítica	app/api/ai/config/route.ts 21	[SEC-02] Chave da API Google Gemini hardcoded e exposta ao público. A chave 'AlzaSy...' está fixa em três pontos: app/api/ai/config/route.ts:21 (servida por GET /api/ai/config SEM autenticação), lib/ai-context.tsx:91 (arquivo 'use client', vai no bundle do navegador) e diagnostico.cjs:2. Também aparece no histórico do git (commits 3c300a2, 16a5416). <i>Impacto:</i> Qualquer visitante lê a chave (via endpoint ou bundle JS) e a usa por conta do titular (custo/abuso).
Alta	lib/ai-context.tsx 11, 91 / lib/actions/config.ts:97	[SEC-03] Chave de IA trafega até o navegador por design. A config de IA (incluindo apiKey) é carregada no cliente, guardada em localStorage e reenviada do navegador para /api/ai/chat no corpo da requisição. Chaves de provedores deveriam permanecer apenas no servidor. <i>Impacto:</i> Exposição contínua da chave a qualquer usuário logado, mesmo sem os hardcodes.

Sev.	Arquivo : linha	Descrição
Média	lib/actions/users.ts 111	[SEC-04] Senha padrão fraca e previsível para novos usuários. Ao criar usuário sem senha, aplica-se <code>bcrypt.hash('123456')</code> . Não há troca obrigatória no primeiro acesso. <i>Impacto:</i> Contas recém-criadas ficam com credencial conhecida até o usuário trocar (se trocar).
Baixa	lib/prisma-db.ts 18	[SEC-05] Ausência de validação de segredos no startup. <code>connectionString</code> cai em fallback silencioso (<code>DB_URL_OFFICIAL DATABASE_URL</code>) e nada rejeita segredos ausentes/padrão na inicialização. <i>Impacto:</i> Falhas de configuração passam despercebidas; defaults inseguros podem ir a produção.

5. Inputs sem tratamento (XSS)

Sev.	Arquivo : linha	Descrição
Baixa	app/layout.tsx 71	[XSS-01] Injeção de CSS via valor de localStorage sem validação. No ramo de gradiente, <code>savedColor</code> (<code>localStorage 'flexo_theme_sidebar'</code>) é concatenado direto em <code>style.innerHTML</code> de um <code><style></code> . O ramo <code>'#'</code> usa <code>setProperty</code> (seguro); o de gradiente não valida. Explorável apenas se o atacante já controla o <code>localStorage</code> da vítima (XSS prévio ou acesso local). <i>Impacto:</i> Injeção de CSS/estilo; risco baixo por depender de controle prévio do armazenamento local.

4. Recomendações priorizadas

Prio.	Ação	Detalhe
P1	Implementar sessão real no servidor	Trocar a sessão em localStorage por cookie httpOnly assinado (ou JWT verificado no servidor). Derivar SEMPRE o usuário/vendedor da sessão do servidor — nunca de um requestId enviado pelo cliente. Adicionar middleware.ts protegendo rotas e server actions. (AUTH-01, TEN-01, IDOR-02/03)
P1	Autorização por papel no servidor em toda operação sensível	Criar um guard requireAdmin()/requireOwner() aplicado no início de saveUser, toggleUserActive, updateUserPassword, saveVendedor, updateEmpresa, updateAIConfig, addMovimentacaoCredito e nas rotas de formas de pagamento. (AUTHZ-01..05)
P1	Rotacionar e remover segredos versionados	Rotacionar imediatamente a senha do banco Neon e a chave Gemini (estão no histórico do git). Remover .env do controle de versão (git rm --cached .env) e adicioná-lo ao .gitignore. Excluir a chave hardcoded de route.ts, ai-context.tsx e diagnostico.cjs. (SEC-01, SEC-02)
P2	Manter a chave de IA apenas no servidor	O endpoint /api/ai/chat deve ler a chave do ambiente/servidor; nunca retornar apiKey ao cliente nem recebê-la do corpo. Proteger GET /api/ai/config com auth e remover a apiKey da resposta. (SEC-03)
P2	Forçar filtro de inquilino no servidor (não opcional)	Aplicar o filtro por vendedorId a partir da sessão em getPedidos, getOrcamentos, getComissoes, getDashboardMetrics, getOportunidadesData, getClienteById/getClientes — sem depender de parâmetro do cliente. (TEN-01, TEN-02, IDOR-01)
P3	Política de credenciais e validação de startup	Remover a senha padrão '123456' (gerar aleatória + troca obrigatória no 1º acesso) e validar variáveis de ambiente na inicialização, rejeitando defaults inseguros. (SEC-04, SEC-05)
P3	Sanitarizar/validar entradas em sinks de HTML/CSS	Validar savedColor contra um padrão seguro (hex/gradiente permitido) antes de injetar no <style>. Adotar uma lib de sanitização caso surja renderização de HTML/markdown. (XSS-01)

5. Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar. Achados triviais relacionados foram agrupados na mesma issue.

Issue 1 — [Segurança] Implementar sessão/autenticação no servidor (fim do requesterId enviado pelo cliente)

```
--- ISSUE 1 ---
**Título:** [Segurança] Implementar sessão/autenticação no servidor (fim do requesterId enviado pelo cliente)

**Labels:** security, critical

## Problema
A aplicação não tem sessão no servidor. A identidade vem de localStorage e de um userId/requesterId enviado pelo cliente; verifySession(userId) confia nesse id. Isso permite se passar por qualquer usuário e é a causa-raiz dos problemas de isolamento, autorização e IDOR.

## Evidência
```
lib/auth-context.tsx:44-135 (sessão em localStorage)
lib/actions/users.ts:147-176 (verifySession recebe userId do cliente)
Ausência de middleware.ts no projeto
```

## Impacto
Bypass total de autenticação e de papéis; base para escalonamento de privilégio e vazamento entre inquilinos.

## Sugestão de correção
Adotar cookie httpOnly assinado (ou JWT verificado). Derivar usuário/papel/vendedor SEMPRE da sessão do servidor. Criar middleware.ts protegendo páginas e um helper getSessionUser() usado pelas server actions.

## Critérios de aceite
- [ ] Sessão emitida em cookie httpOnly + Secure + SameSite
- [ ] Nenhuma server action aceita userId/requesterId do cliente para autorização
- [ ] middleware.ts redireciona não autenticados
- [ ] verifySession não recebe mais o id pelo cliente
--- FIM ISSUE 1 ---
```


Issue 2 — [Segurança] Verificação de papel (admin/owner) no servidor para operações privilegiadas

--- ISSUE 2 ---

****Título:**** [Segurança] Verificação de papel (admin/owner) no servidor para operações privilegiadas

****Labels:**** security, critical

Problema

Operações administrativas e de escrita são protegidas apenas escondendo a UI (isAdmin) no frontend. As server actions correspondentes não validam papel e são invocáveis diretamente.

Evidência

```

lib/actions/users.ts:93,127,140 (saveUser/toggleUserActive/updateUserPassword)

lib/actions/vendedores.ts:84,110

lib/actions/config.ts:78,117

lib/actions/creditos.ts:6,49

app/api/formas-pagamento/[id]/route.ts:4,31

Gates de UI: app/usuarios/page.tsx:80, app/configuracoes/page.tsx:139,

app/vendedores/page.tsx:64, app/formas-pagamento/page.tsx:149

```

Impacto

Não-admin cria admin, redefine senhas (account takeover), altera comissões, config da empresa e crédito financeiro.

Sugestão de correção

Implementar requireAdmin() e requireOwner() a partir da sessão do servidor e aplicá-los no início de cada operação sensível listada.

Critérios de aceite

- [] Toda action de gestão chama requireAdmin() antes de qualquer escrita
- [] Operações por dono validam vínculo do recurso com o usuário da sessão
- [] Testes cobrindo chamada direta por não-admin retornando 403/erro

--- FIM ISSUE 2 ---

Issue 3 — [Segurança] Isolamento de inquilino (vendedor) forçado no servidor e não bypassável

```

--- ISSUE 3 ---
**Título:** [Segurança] Isolamento de inquilino (vendedor) forçado no servidor e não
bypassável

**Labels:** security, critical

## Problema
O filtro por vendedorId é opcional e depende de requesterId enviado pelo cliente. Sem esse
parâmetro, listas e agregações retornam dados de todos os vendedores. Nas leituras/escritas
por id de pedidos e orçamentos, a checagem de posse também some quando requesterId não é
enviado.

## Evidência
```
lib/actions/pedidos.ts:47,61-63,213,246,287,346
lib/actions/orçamentos.ts:34-39,181,215,270,287
lib/actions/comissoes.ts:11-16
lib/actions/dashboard.ts:16-21
lib/actions/oportunidades.ts:16-21
```

## Impacto
Vendedor ou anônimo vê/edita pedidos, orçamentos, comissões e faturamento de toda a empresa.

## Sugestão de correção
Derivar o vendedorId da sessão e aplicá-lo incondicionalmente. Remover o caminho 'sem filtro'
quando não-admin. A checagem de posse deixa de depender de parâmetro opcional.

## Critérios de aceite
- [ ] Filtro por vendedorId aplicado a partir da sessão, sem parâmetro do cliente
- [ ] Requisição sem 'requesterId' não retorna dados de outros vendedores
- [ ] getPedidoById/getOrçamentoById retornam null para recurso de outro vendedor sempre
--- FIM ISSUE 3 ---

```

Issue 4 — [Segurança] IDOR na base de clientes (leitura/escrita por id sem posse)

```

--- ISSUE 4 ---
**Título:** [Segurança] IDOR na base de clientes (leitura/escrita por id sem posse)

**Labels:** security, high

## Problema
Funções de cliente operam por id sem verificar contexto do chamador. getClienteById devolve
cadastro, saldos e todo o histórico (inclusive de outros vendedores); saveCliente edita
qualquer cliente.

## Evidência
```
lib/actions/clientes.ts:115-149 (getClienteById + orçamentos/pedidos)
lib/actions/clientes.ts:170 (saveCliente)
lib/actions/creditos.ts:49 (getMovimentacoesByCliente)
```

## Impacto
Enumeração completa da base de clientes, leitura de dados financeiros e alteração de
cadastros.

## Sugestão de correção
Exigir sessão e aplicar regra de acesso (admin vê tudo; vendedor só clientes com relação a
ele). Filtrar o histórico retornado pelo escopo do usuário.

## Critérios de aceite
- [ ] getClienteById exige sessão e restringe histórico ao escopo do usuário
- [ ] saveCliente valida permissão antes de gravar
- [ ] getMovimentacoesByCliente restrito por escopo
--- FIM ISSUE 4 ---

```

Issue 5 — [Segurança] Segredos versionados: credenciais do banco e chave da API Gemini

--- ISSUE 5 ---

****Título:**** [Segurança] Segredos versionados: credenciais do banco e chave da API Gemini

****Labels:**** security, critical

Problema

A senha do banco Neon está em .env versionado e a chave Gemini está hardcoded em três arquivos – um deles servido por GET sem auth e outro no bundle do cliente. Ambos aparecem no histórico do git.

Evidência

```
`.env:1 (DB_URL_OFFICIAL com senha real; .env rastreado no git)
app/api/ai/config/route.ts:21 (chave via GET público)
lib/ai-context.tsx:91 ('use client' → bundle)
diagnostico.cjs:2
Histórico: commits 3c300a2, 16a5416
`
```

Impacto

Acesso direto ao banco de produção e uso indevido da chave de IA por terceiros.

Sugestão de correção

Rotacionar senha do banco e chave Gemini. `git rm --cached .env` e adicionar ao `.gitignore`. Remover a chave dos três arquivos. Considerar limpeza do histórico (`filter-repo/BFG`). Chave de IA só no servidor.

Critérios de aceite

- [] Senha do banco e chave Gemini rotacionadas
 - [] .env fora do versionamento e no .gitignore
 - [] Nenhuma chave hardcoded no código ou no bundle do cliente
 - [] GET /api/ai/config protegido e sem apiKey na resposta
- FIM ISSUE 5 ---

Issue 6 — [Segurança] Chave de IA deixa de trafegar até o navegador

--- ISSUE 6 ---

****Título:**** [Segurança] Chave de IA deixa de trafegar até o navegador

****Labels:**** security, high

Problema

A configuração de IA (com apiKey) é carregada no cliente, salva em localStorage e reenviada do navegador para /api/ai/chat no corpo da requisição.

Evidência

```
lib/ai-context.tsx:11,91
lib/actions/config.ts:97-138
app/api/ai/chat/route.ts:223 (apiKey vem do body)
components/ai-chat-panel.tsx:116 / app/oportunidades/page.tsx:133 (envio pelo cliente)
`
```

Impacto

Exposição contínua da chave a qualquer usuário do app, mesmo após remover os hardcodes.

Sugestão de correção

/api/ai/chat lê a chave de variável de ambiente no servidor; remover apiKey do contexto do cliente e do body.

Critérios de aceite

- [] apiKey nunca é enviada ao cliente nem recebida no body
 - [] Chamadas de IA usam chave do ambiente do servidor
- FIM ISSUE 6 ---

Issue 7 — [Segurança] Senha padrão fraca para novos usuários

--- ISSUE 7 ---

****Título:**** [Segurança] Senha padrão fraca para novos usuários

****Labels:**** security, medium

Problema

Novos usuários sem senha recebem '123456' e não há troca obrigatória no primeiro acesso.

Evidência

```

lib/actions/users.ts:111 (bcrypt.hash('123456'))

```

Impacto

Contas novas ficam com credencial previsível até (e se) o usuário trocar.

Sugestão de correção

Gerar senha aleatória forte e exigir redefinição no primeiro login; nunca usar constante.

Critérios de aceite

- [] Novo usuário recebe senha aleatória (ou convite por link)
- [] Flag de troca obrigatória no primeiro acesso

--- FIM ISSUE 7 ---

Issue 8 — [Segurança] Injeção de CSS via valor de tema em localStorage

--- ISSUE 8 ---

****Título:**** [Segurança] Injeção de CSS via valor de tema em localStorage

****Labels:**** security, low

Problema

No ramo de gradiente, savedColor de localStorage é concatenado sem validação em style.innerHTML.

Evidência

```

app/layout.tsx:62-72 (ramo gradiente concatena savedColor em <style>.innerHTML)

```

Impacto

Injeção de CSS; risco baixo por exigir controle prévio do localStorage da vítima.

Sugestão de correção

Validar savedColor contra um padrão permitido (hex ou gradiente conhecido) antes de aplicar.

Critérios de aceite

- [] savedColor validado por regex/allowlist antes de injetar
- [] Valores inválidos são ignorados com fallback seguro

--- FIM ISSUE 8 ---